

Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales con optimizaciones específicas para hardware NVIDIA

Alumno: Haro Armero, Abel

Tutor: Flich Cardo, José

Co-tutor: López Rodríguez, Pedro Juan

Fecha: 15 de julio de 2025

1. Introducción
2. Motivación
3. Objetivos
4. Conceptos previos
5. Propuesta de solución
6. Desarrollo de la solución
7. Demo del sistema
8. Resultados
9. Conclusiones

1. Introducción

- Crecimiento exponencial de la **Inteligencia Artificial**.
- Avances en **visión por computador** gracias a las **CNNs**.
- Restricciones **temporales** en el acceso a centros de datos.
- Dispositivos optimizados para el **deep learning** en el **edge**.

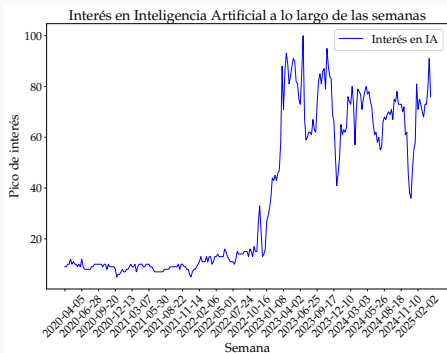


Figura 1: Interés en IA y visión por computador.

2. Motivación

- La inspección **manual** de **defectos** es un proceso **lento** y propenso a **errores**.
- Emular la **interpretación humana** de imágenes en máquinas.
- Automatización de la **detección y clasificación** de objetos en movimiento en la industria.



Figura 2: Control de calidad en la industria.

3. Objetivos

Objetivo Principal: Desarrollar un sistema de **detección de defectos** en objetos en movimiento utilizando CNNs, optimizado para hardware **NVIDIA**.

Objetivos Específicos:

1. **Investigación:** Estado del arte de **CNNs** y **aceleradores hardware** (NVIDIA Jetson).
2. **Datos:** Crear un **conjunto de datos** representativo de objetos con/sin defectos.
3. **Modelos:** Entrenar y optimizar **modelos CNN** para la detección en **tiempo real**.
4. **Integración:** Creación de un **sistema de visión artificial**.
5. **Optimización:** Identificar **cuellos de botella** para mejorar el rendimiento.
6. **Evaluación:** Evaluar el sistema con métricas clave de **precisión, latencia y consumo**.
7. **Análisis Comparativo:** Determinar la configuración óptima del sistema.

4. Conceptos Previos - Redes Neuronales Convolucionales (CNNs)

- **CNNs**: redes neuronales para datos matriciales, como imágenes.
- Detectan y clasifican **objetos/defectos**.
- Usan capas **convolucionales** para extraer características.
- Tipos: **dos etapas** (R-CNN) y **una etapa** (YOLO).

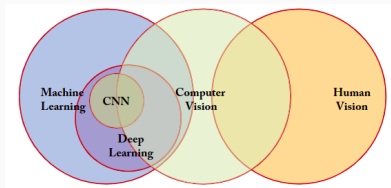


Figura 3: Diagrama de Venn de IA, Machine Learning y Deep Learning.

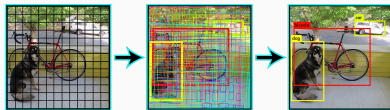


Figura 4: Ejemplo de detección de objetos con YOLO.

4. Conceptos Previos - Hardware NVIDIA Jetson

- Aceleradores hardware para **deep learning** en el **edge**.
- **NVIDIA Jetson: SoC** (CPU, GPU y DLA), dispositivos equilibrados en **potencia de cómputo, consumo y coste**.
- **Jetpack SDK y TensorRT**: Herramientas para **crear y optimizar** redes neuronales.

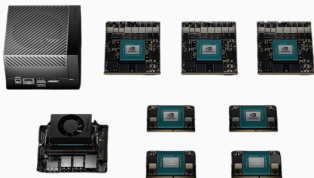


Figura 5: Familia de dispositivos NVIDIA Jetson.

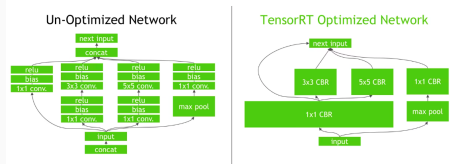


Figura 6: Ejemplo de optimizaciones de TensorRT.

4. Conceptos Previos - Seguimiento de objetos (MOT)

- **Seguimiento de objetos:** Identifica y sigue la **trayectoria** de un objeto en el tiempo.
- Combina **detección** y **análisis de movimiento**.
- **BYTETrack:** Algoritmo de seguimiento a partir de las **detecciones**.

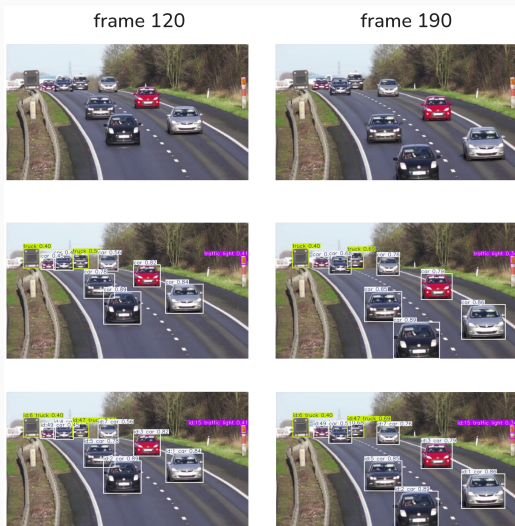


Figura 7: Ejemplo de seguimiento de objetos con BYTETrack.

5. Propuesta de solución

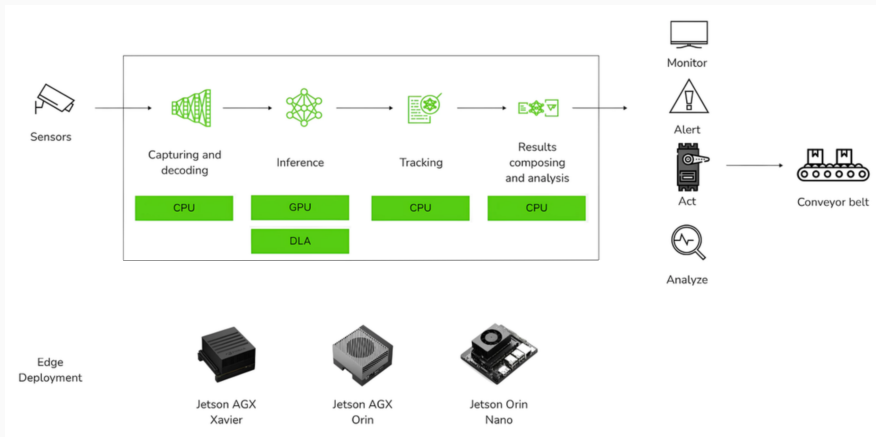


Figura 8: Figura de la solución propuesta.

6. Desarrollo de la solución - Creación del conjunto de datos

- **Objetivo:** Crear un conjunto de datos representativo de objetos en movimiento con defectos.
- **Problema:** Difícil acceso a un dataset real de objetos en movimiento con defectos.
- Utilización de un entorno simulado con **canicas** de distintos **colores** y **defectos**.
- +600 imágenes con **anotaciones de clase** y **bounding boxes**.



Figura 9: Ejemplo de canicas del conjunto de datos.

6. Desarrollo de la solución - Entrenamiento y validación de modelos

- Entrenamiento de modelos **CNN** para **detección de defectos**.
- Validación y ajuste de **hiperparámetros** para mejorar **precisión** (+100 entrenamientos).
- Exportación de los modelos a **TensorRT** para optimizar su rendimiento.

Modelo	Parámetros (M)
YOLOv5-nu	2.6
YOLOv5-mu	25.1
YOLOv8-n	3.2
YOLOv8-s	11.2
YOLO11-n	2.6
YOLO11-s	9.4
YOLO11-m	20.1
YOLO11-l	25.3
YOLO11-x	56.9

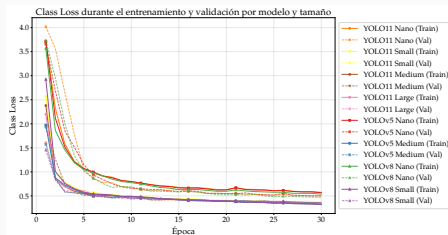


Figura 10: Curva de *loss* de todos los modelos entrenados.

6. Desarrollo de la solución - Segmentación de las etapas (1/6)

- El sistema se divide en cuatro etapas / módulos:
 - Captura** de vídeo.
 - Detección** de objetos.
 - Seguimiento** de objetos.
 - Escritura** de resultados.
- Problema:** El procesamiento **secuencial impide** la **ejecución en tiempo real** (30 FPS). El siguiente frame de vídeo **NO se procesa** hasta que el anterior **ha finalizado**, a pesar de que las etapas son **independientes**.

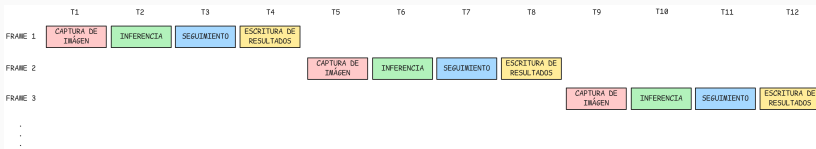


Figura 11: Procesamiento secuencial de las etapas.

6. Desarrollo de la solución - Segmentación de las etapas (2/6)

- **Solución: Segmentar las etapas** para que se ejecuten de forma independiente y concurrente.
- Segmentaciones implementadas: (1) hilos (2) procesos (3) **procesos con memoria compartida** (4) heterogénea.

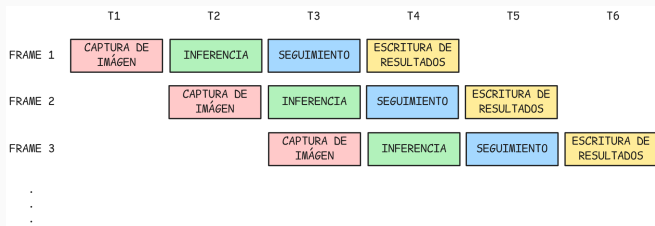
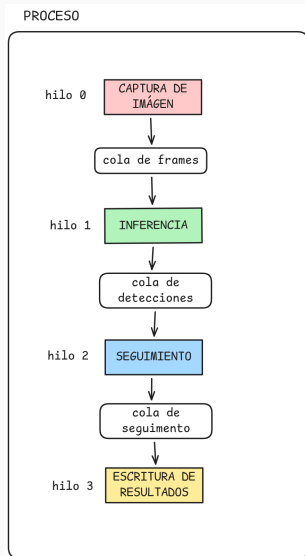


Figura 12: Procesamiento de las etapas segmentadas.

6. Desarrollo de la solución - Segmentación de las etapas en Hilos (3/6)

1. Segmentación por hilos:

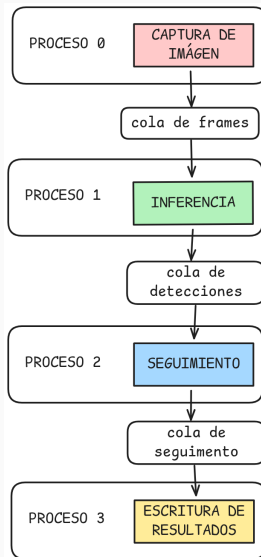
- Cada etapa se ejecuta en un **hilo separado**.
- ↑ **Comunicación** entre etapas **rápida y eficiente** al compartir la misma **memoria**.
- ↓ Permite un procesamiento paralelo pero **NO concurrente** debido al GIL (Global Interpreter Lock) de Python.



6. Desarrollo de la solución - Segmentación de las etapas en Procesos (4/6)

2. Segmentación por procesos:

- Cada etapa se ejecuta en un **proceso separado**.
- ↑ Permite un **procesamiento concurrente** al evitar el GIL de Python.
- ↓ La **comunicación** entre procesos es **más lenta** y **costosa** al requerir mecanismos de IPC (Inter-Process Communication).



6. Desarrollo de la solución - Segmentación de las etapas (5/6)

3. Segmentación por procesos con memoria compartida:

- Cada etapa se ejecuta en un proceso separado, donde **comparten memoria**.
- ↑ Permite un procesamiento **concurrente** con **menor latencia** en la comunicación.
- ↑ Es la que mejor rendimiento ofrece en términos de **latencia** y **consumo**.

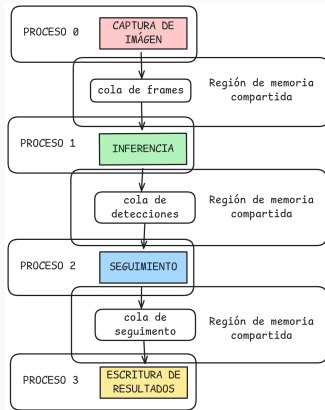
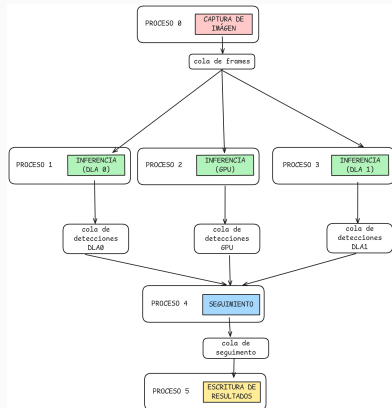


Figura 13: Segmentación por procesos con memoria compartida.

6. Desarrollo de la solución - Segmentación de las etapas Heterogénea (6/6)

4. Segmentación heterogénea:

- Se ejecuta mediante **procesos con memoria compartida**.
- Ejecuta la etapa de de detección en la **GPU o DLAs** del dispositivo Jetson.
- ↑ Teóricamente permite **explotar todos los recursos** para mejorar el rendimiento.
- ↓ Los modelos de no se ejecutan completamente en la **DLA**.



6. Desarrollo de la solución - Prueba de concepto

- Construcción de una **cinta transportadora** para una **prueba de concepto**.
- Se utiliza un **motor**, una **Raspberry Pi Pico** y un **servo** para **rechazar objetos defectuosos**.
- El objetivo es validar que el sistema es capaz de **detectar objetos en movimiento** y **rechazarlos en tiempo real**.

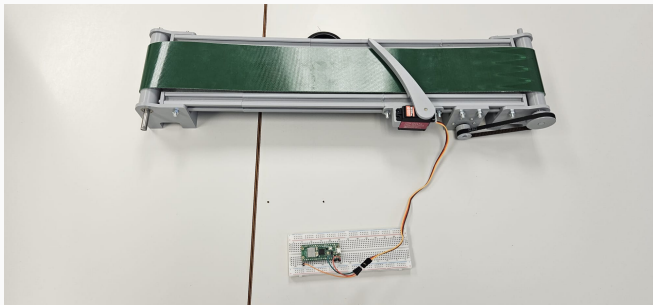


Figura 14: Construcción de la cinta transportadora.

7. Demo del sistema



Figura 15: Vídeo de la demo del sistema.

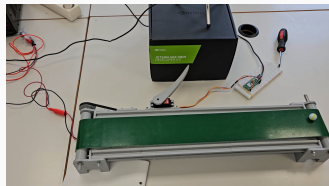


Figura 16: Vídeo de la demo del sistema.

8. Resultados - Metodología

- Evaluación del sistema en términos de **precisión**, **latencia** y **consumo**.
 - Se definen **6 variables** experimentales. En cada prueba, se fija una configuración base y se varía solo una.
1. **Objetos por vídeo:** {17, 43, 84, variable}
 2. **Segmentación:** {hilos, procesos, proc. mem. compartida, heterogénea}
 3. **Modelo y talla:** {YOLOv5{nu, mu}, YOLOv8{n, s}, YOLO11{n}, s, m, l}
 4. **Precisión:** {FP32, FP16, INT8}
 5. **Modo energía dispositivo:** {MAXN, 30W8core, 30W6core, 30W4core, 15W4core}
 6. **Dispositivo:** {AGX Xavier, Orin Nano, AGX Orin}

En total, existen **7200 configuraciones** experimentales posibles.

8. Resultados - Cantidad de objetos

- Evaluación con diferentes **cantidades de objetos**: 17, 43, 84 y carga variable (0-180).
- Se **mantiene** la configuración del resto de **parámetros**, variando únicamente la **cantidad de objetos** presentes en el vídeo.
- El sistema procesa en tiempo real (**30 FPS**) hasta **100 objetos**.

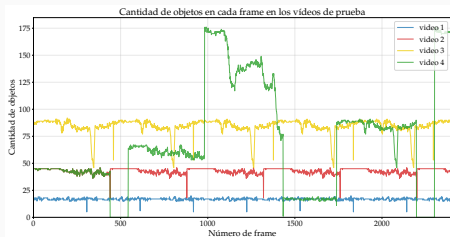


Figura 17: Cantidad de objetos en los vídeos de prueba.

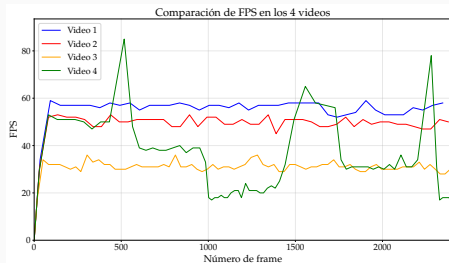


Figura 18: FPS en los vídeos de prueba.

8. Resultados - Tipo de segmentación

- Se ha evaluado el rendimiento del sistema con los diferentes tipos de **segmentación**.
- Se **mantiene** la configuración del resto de **parámetros**, variando únicamente la **segmentación**.
- La segmentación por **procesos con memoria compartida** es la más eficiente, con un **speedup** de 2.47x respecto a la secuencial.

Segmentación	Frames	Tiempo (s) ↓	FPS ↑	LFPS ↓	Potencia (W) ↓	Energía (J) ↓	Frames /W ↓	Speedup ↑	GPU (%) ↓	CPU (%) ↓
Secuencial	2400	198.57	12.09	0.00	9.488	1884.03	1.274	1.000	24.62	15.46
Hilos	2400	131.40	18.26	0.00	12.991	1703.12	1.409	1.511	10.35	25.53
Multiprocesos	2400	105.16	22.82	0.00	14.272	1500.58	1.599	1.888	12.84	33.65
MP Mem. Comp.	2400	80.27	29.90	0.00	16.551	1320.25	1.818	2.474	17.12	40.10
Heterogénea	2400	84.36	28.45	0.00	17.812	1502.39	1.597	2.354	10.00	44.25

Cuadro 1: Resultados comparativos de los distintos modos de segmentación.

9. Conclusiones – Cumplimiento de objetivos (1/2)

Los objetivos del TFG se han alcanzado con éxito:

- **Estudio del estado del arte:** Se realizó un análisis exhaustivo de **CNNs**, **aceleradores hardware** y plataformas **NVIDIA Jetson**, superando desafíos de compatibilidad y configuración.
- **Creación del conjunto de datos:** Se generó un **dataset** y vídeos de entrenamiento que simulan condiciones reales de operación con objetos de variabilidad controlada.
- **Entrenamiento de modelos CNN:** Se entrenaron múltiples modelos de las familias **YOLOv5**, **YOLOv8** y **YOLO11** en diversas tallas, optimizando hiperparámetros mediante iteraciones exhaustivas.

9. Conclusiones – Cumplimiento de objetivos (2/2)

- **Implementación del sistema integrado:** Se desarrolló un sistema de **visión artificial** que combina **detección** con **seguimiento** BYTETrack, manteniendo la identidad de objetos a lo largo del tiempo.
- **Análisis y optimización de cuellos de botella:** Se exploraron estrategias de segmentación (**hilos, procesos, memoria compartida, heterogénea**), identificando la **segmentación por procesos con memoria compartida** como la más efectiva.
- **Evaluación con métricas completas:** Se realizaron experimentos exhaustivos evaluando **precisión, latencia y consumo** en diferentes configuraciones.

Detección de defectos en objetos en movimiento mediante Redes Neuronales Convolucionales con optimizaciones específicas para hardware NVIDIA

Alumno: Haro Armero, Abel

Tutor: Flich Cardo, José

Co-tutor: López Rodríguez, Pedro Juan

Fecha: 15 de julio de 2025